

decorator to the new function, and add the name of this function as a parameter for each test function that uses this resource. Your tests will be shorter and easier to write and maintain from that point forward.

TRY IT YOURSELF

11-3. Employee: Write a class called `Employee`. The `__init__()` method should take in a first name, a last name, and an annual salary, and store each of these as attributes. Write a method called `give_raise()` that adds \$5,000 to the annual salary by default but also accepts a different raise amount.

Write a test file for `Employee` with two test functions, `test_give_default_raise()` and `test_give_custom_raise()`. Write your tests once without using a fixture, and make sure they both pass. Then write a fixture so you don't have to create a new employee instance in each test function. Run the tests again, and make sure both tests still pass.

Summary

In this chapter, you learned to write tests for functions and classes using tools in the `pytest` module. You learned to write test functions that verify specific behaviors your functions and classes should exhibit. You saw how fixtures can be used to efficiently create resources that can be used in multiple test functions in a test file.

Testing is an important topic that many newer programmers aren't exposed to. You don't have to write tests for all the simple projects you try as a new programmer. But as soon as you start to work on projects that involve significant development effort, you should test the critical behaviors of your functions and classes. You'll be more confident that new work on your project won't break the parts that work, and this will give you the freedom to make improvements to your code. If you accidentally break existing functionality, you'll know right away, so you can still fix the problem easily. Responding to a failed test that you ran is much easier than responding to a bug report from an unhappy user.

Other programmers will respect your projects more if you include some initial tests. They'll feel more comfortable experimenting with your code and be more willing to work with you on projects. If you want to contribute to a project that other programmers are working on, you'll be expected to show that your code passes existing tests and you'll usually be expected to write tests for any new behavior you introduce to the project.

Play around with tests to become familiar with the process of testing your code. Write tests for the most critical behaviors of your functions and classes, but don't aim for full coverage in early projects unless you have a specific reason to do so.